



第16章 SQLite简介

清华大学计算机系 李国良

liguoliang@tsinghua.edu.cn

<http://dbgroup.cs.tsinghua.edu.cn/ligl>



目录

01

SQLite嵌入式数据库

02

SQLite总体架构

03

SQLite查询处理技术

04

SQLite存储技术

05

SQLite事务管理技术



目录

01

SQLite嵌入式数据库

02

SQLite总体架构

03

SQLite查询处理技术

04

SQLite存储技术

05

SQLite事务管理技术



嵌入式数据库产生的背景

- ❑ 嵌入式系统是指将软件嵌入到应用产品内部的一种专用计算机系统，在各种应用场景中起着非常重要的作用
- ❑ 互联网技术的不断发展，数据库的用户数量与日俱增，数据量级的不断提升
- ❑ 传统的关系型数据库由于运行缓慢、性能要求较高等问题并不适用于应用场景苛刻的嵌入式系统
- ❑ 微处理器与数据存储技术得到了高速的发展，硬件的性能有了显著的提升
- ❑ 因此，嵌入式系统专用的嵌入式数据库应运而生
 - 手机、机顶盒、电视、游戏机、照相机、手表、厨房用具、恒温器等



嵌入式数据库的特点

□ 嵌入性

- 既可以嵌入至应用程序，也可以嵌入至嵌入式设备

□ 移植性

- 支持不同操作系统

□ 占用存储空间小

□ 实时性

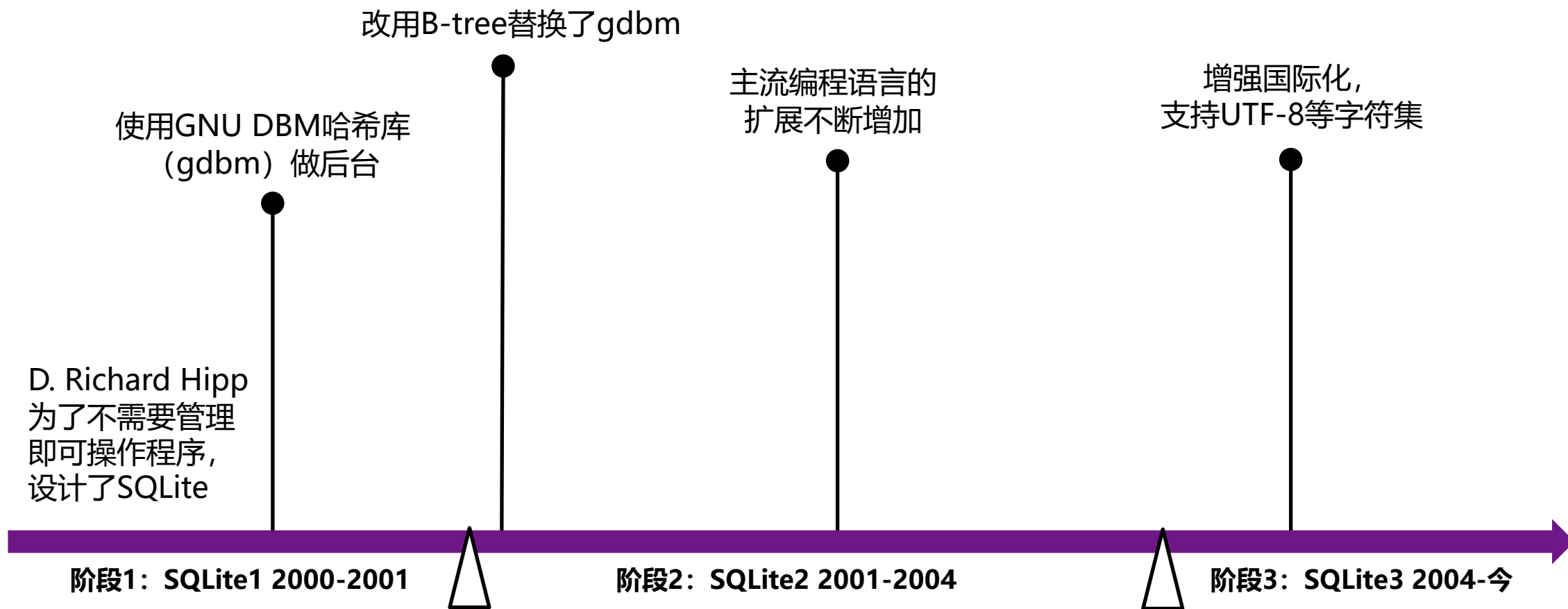
- 在规定时间内快速完成指定操作

□ 可靠性

- 具有一定的自我维护机制，满足苛刻环境的需要



SQLite数据库发展历史



数十亿装机量, 功能齐全的SQL, ACID, 轻量, 单个文件的数据库



SQLite数据库的特点

□ 零配置

- 不使用配置文件，无需配置数据库实例分配权限、运行故障可自行恢复

□ 移植性

- 支持多种操作系统与嵌入式平台
- 数据库文件均为兼容二进制

□ 紧凑性

- 包内只有一个头文件与库文件

□ 简单性

□ 灵活性

- 具有精心设计的前端与后端

□ 自由授权

□ 可靠性

- 源码撰写规范标准，具有100%测试覆盖率

□ 易用性，免配置



SQLite嵌入式的使用场景

- 发布至今，SQLite现在已经是全球用户最多的数据库产品
- 嵌入式设备和物联网
 - 手机、机顶盒、电视、游戏机、照相机、手表、厨房用具、恒温器等
- 网站
 - 对于大多数网站而言，SQLite可以完美胜任数据库引擎任务
- 数据分析
- 缓存企业数据
- 服务器端数据库
 - 使用SQLite作为特定于应用程序的数据库服务器的基础存储引擎



目录

01

SQLite嵌入式数据库

02

SQLite总体架构

03

SQLite查询处理技术

04

SQLite存储技术

05

SQLite事务管理技术



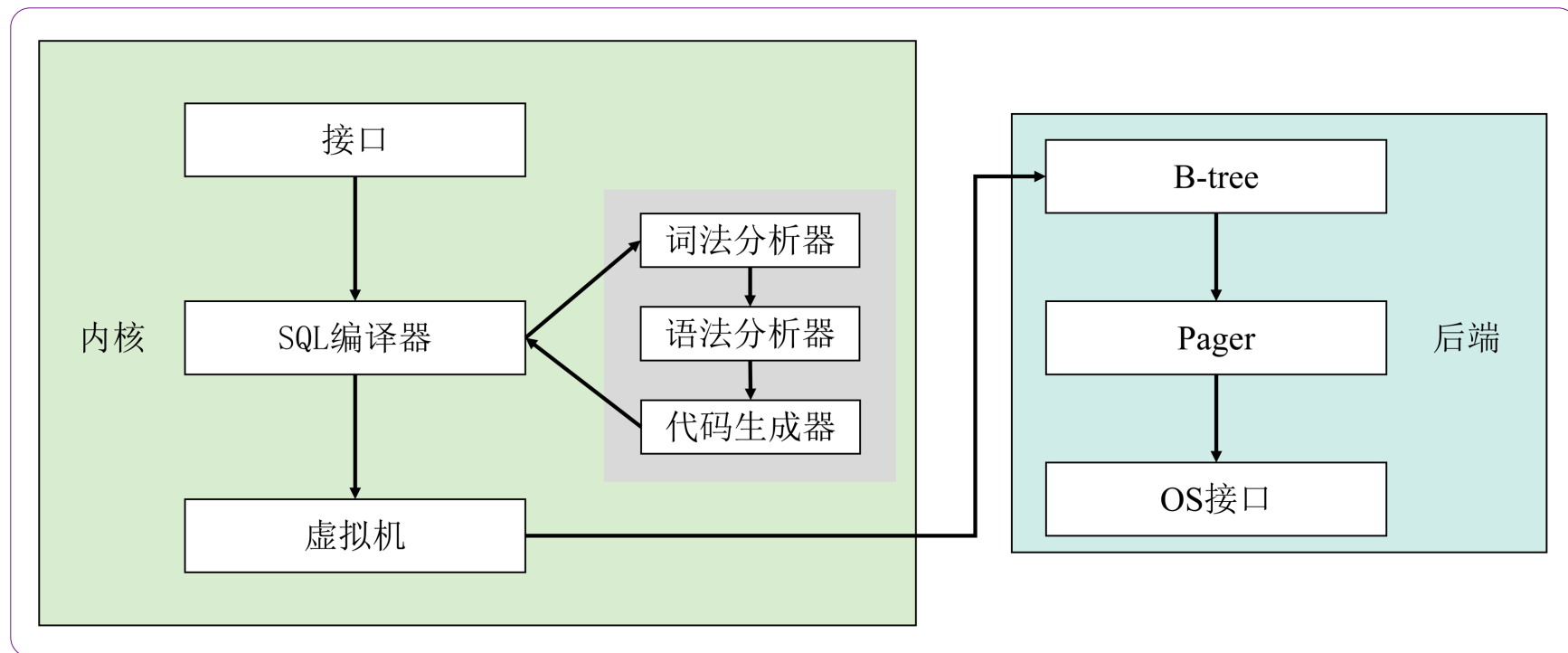
SQLite总体架构

□ 内核

- 接口
- SQL编译器
 - 词法分析器
 - 语法分析器
 - 代码生成器
- 虚拟机

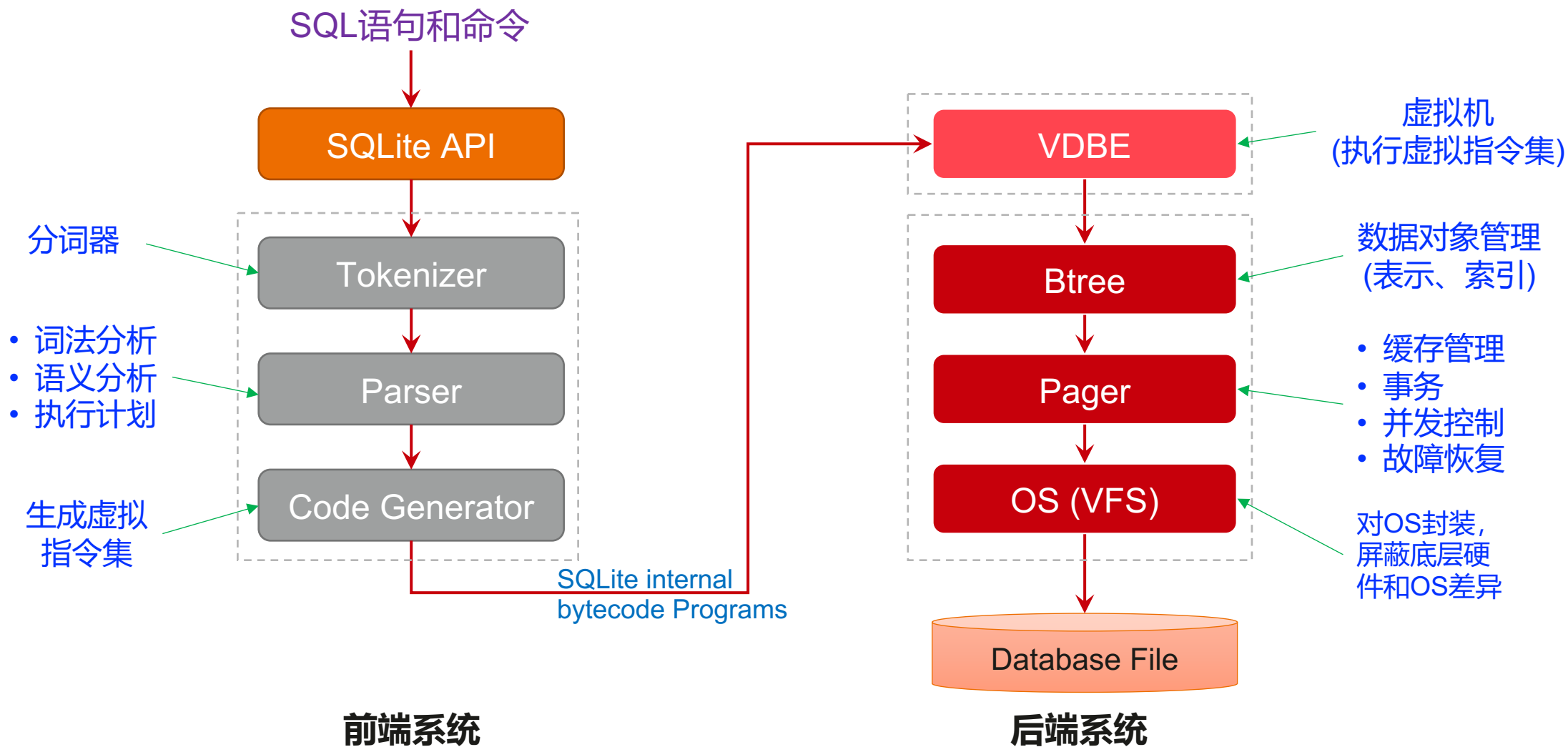
□ 后端

- B-Tree
- Pager
- OS接口





SQLite总体架构



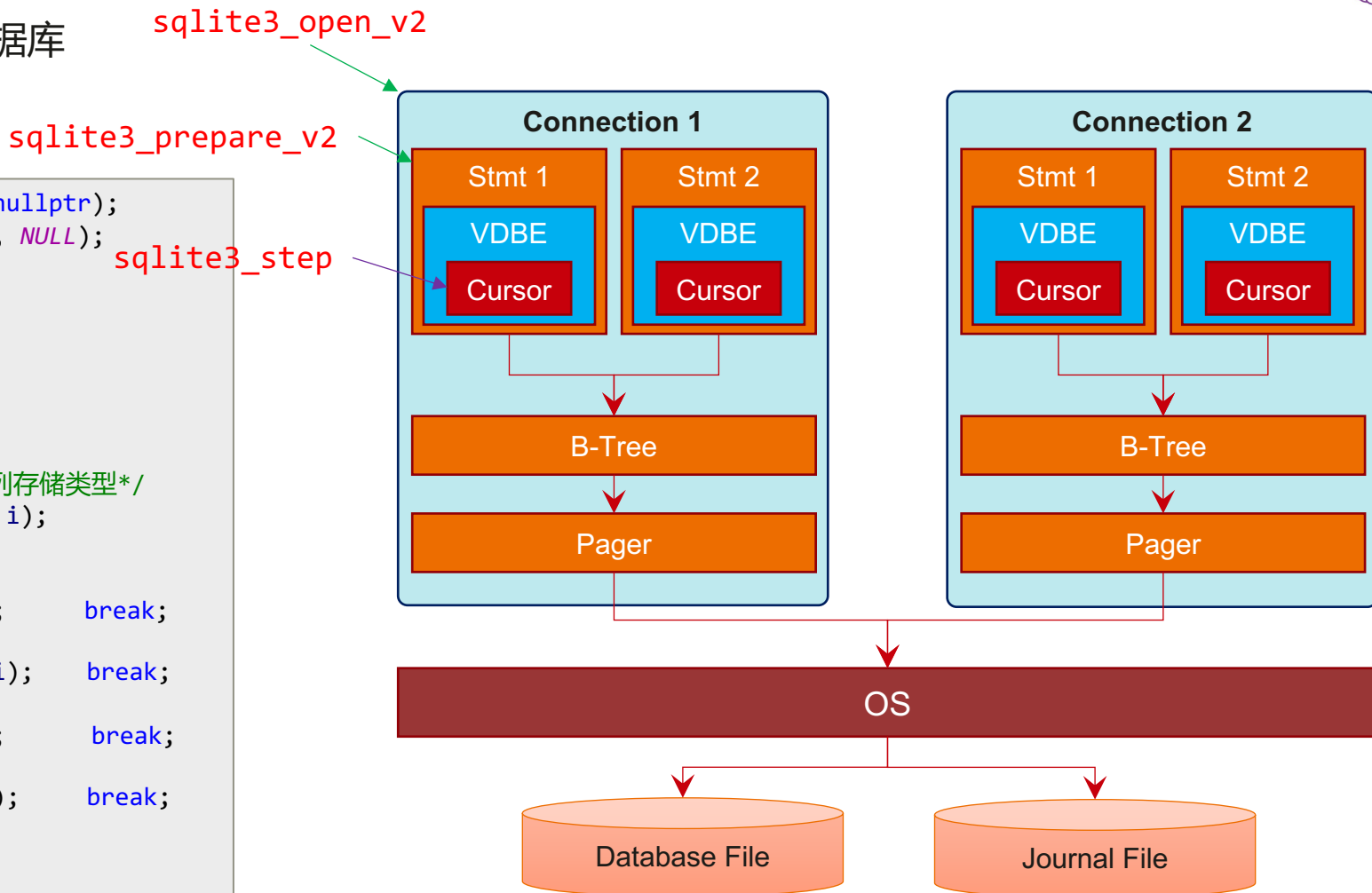


SQLite运行视图

- 一个数据库可以创建多个连接同时操作数据库
- 每个连接可以创建多个Statement
- 使用sqlite3_step控制游标的位置

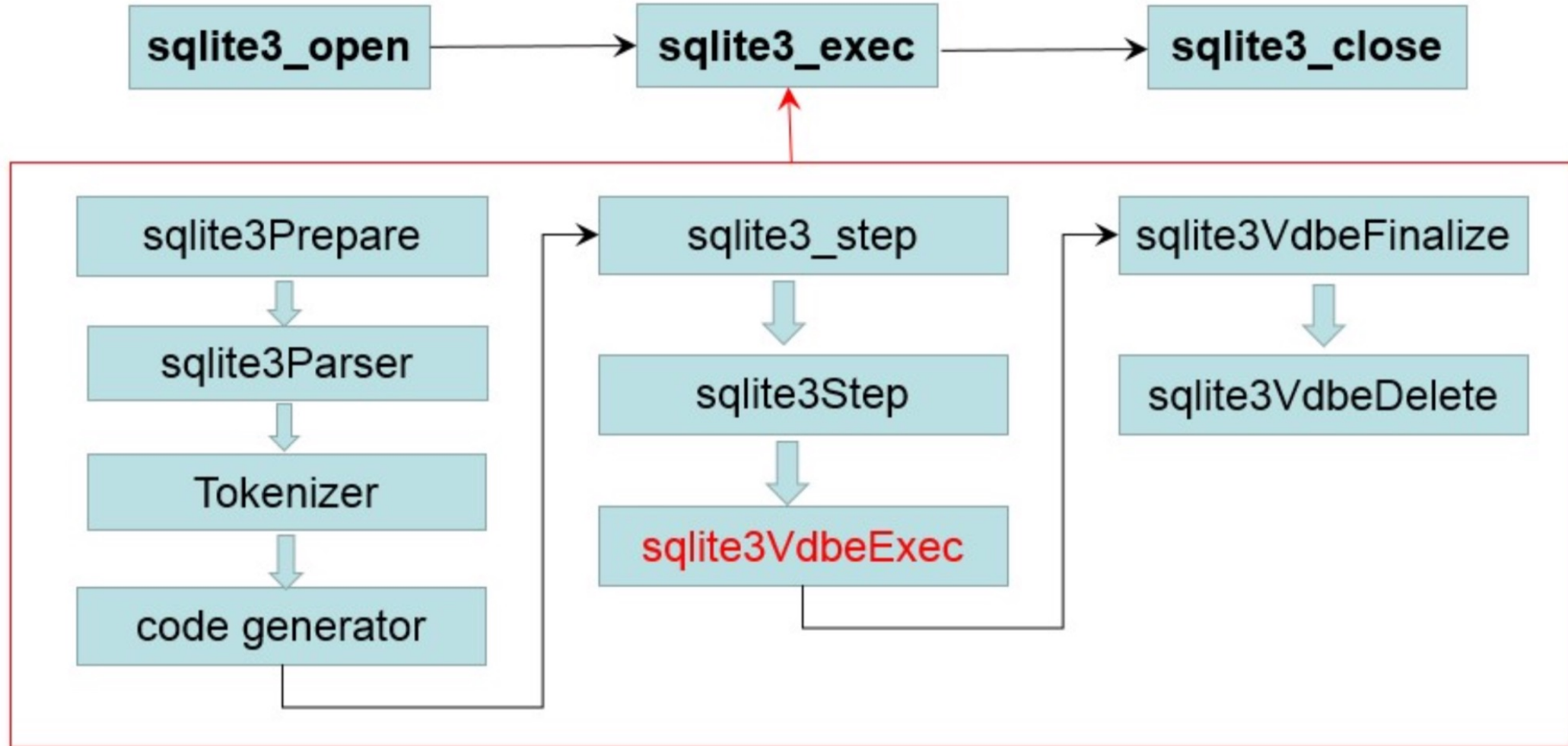
```
int rc = sqlite3_open_v2(dbURL, &pDb, openFlag, nullptr);
int rc = sqlite3_prepare_v2(pDb, sql, -1, &pStmt, NULL);

//获取列数目
int n_columns = sqlite3_column_count(pStmt);
do {
    ret = sqlite3_step(stmt);
    if (ret == SQLITE_ROW) {
        //处理每一列
        for (i = 0; i < n_columns; i++) { /*获取列存储类型*/
            int type = sqlite3_column_type(stmt, i);
            switch (type) {
                case SQLITE_INTEGER: /*处理整型*/
                    sqlite3_column_int(stmt, i); break;
                case SQLITE_FLOAT: /*处理浮点数*/
                    sqlite3_column_double(stmt, i); break;
                case SQLITE_TEXT: /*处理字符串*/
                    sqlite3_column_text(stmt, i); break;
                case SQLITE_BLOB: /*处理二进制*/
                    sqlite3_column_blob(stmt, i); break;
                case SQLITE_NULL: /*处理空*/
                    break;
            }
        }
    } else if (ret == SQLITE_DONE) { //结束
        break;
    }
} while (true);
```





SQLite执行过程





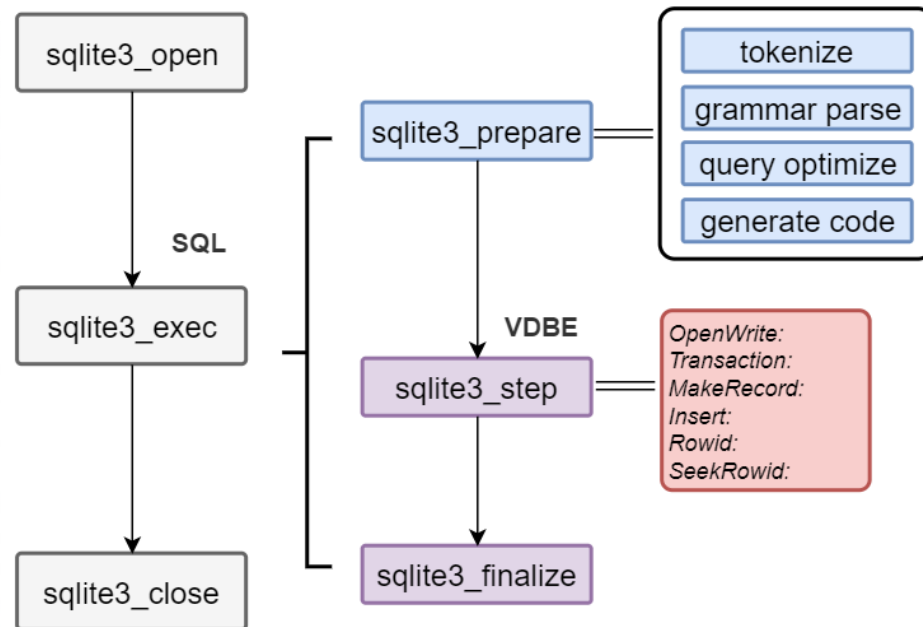
SQLite总体架构

□ 接口

- 由SQLite C API组成
- 各种程序最终都要通过接口与SQLite进行通信

□ SQL编译器

- SQL编译器可以将SQL语句编译成字节码程序
- SQL编译器包含词法分析器、语法分析器、代码生成器
 - 包含SQL语句的字符串首先被发送到词法分析器，词法分析器对其进行分析得到一系列tokens，并将这些tokens逐一传递给语法分析器
 - 语法分析器根据tokens的上下文赋予其对应的意义，并将tokens重组成语法树（parse tree）形式，然后将该树传递给代码生成器
 - 代码生成器分析语法树，生成一种SQLite专用的字节码（bytecode）程序，并交给虚拟机运行





SQLite总体架构

□ 虚拟机

- 负责运行代码生成器传递过来的字节码程序

□ 后端

- 负责处理存储和与操作系统进行交互
- 后端包含B-tree、Pager、OS接口
 - B-tree将包含字段或索引项等信息的页组织成适合搜索的树状结构，并且以固定页面大小从磁盘请求信息
 - Pager负责根据B-tree的请求读取、写入和缓存页。Pager还有事务管理功能，可以进行回滚、原子提交和故障恢复等操作，并提供锁机制来锁定数据库文件
 - OS接口可以为其他模块提供屏蔽操作系统差异的抽象层。
 - SQLite使用一个名为VFS的抽象对象来提供跨操作系统的可移植性，每个VFS都提供了打开、读取、写入和关闭磁盘上文件的方法，以及其他特定于操作系统的任务，例如查找当前时间。其他模块无需关心这些任务是如何在某个操作系统上实现的



目录

01

SQLite嵌入式数据库

02

SQLite总体架构

03

SQLite查询处理技术

04

SQLite存储技术

05

SQLite事务管理技术



SQLite查询分析



□ 词法分析

- 循环分析用户输入的SQL查询字符串，得到字符串中的token及其他信息，交由语法分析处理

□ 语法分析

- SQLite的语法分析器是由Lemon生成的LALR(1)分析器，根据LR(1)分析表对由词法分析得到的token构建语法树

□ 语义分析

- 根据语法树对给定查询的语义方面进行检查，从数据字典中找到有关模式定义的信息，检查语句中的数据库对象（关系、属性）是否存在和有效



SQLite查询优化

□ 基于规则的查询优化 (RBO)

- 根据预定义的启发式规则对SQL语句进行优化

□ 基于代价的查询优化 (CBO)

- 对SQL语句对应的待选物理计划进行代价估算，从待选计划中选择代价最低的物理计划作为最终的执行计划

□ 基于人工智能的查询优化 (ABO)

- 对SQL语句对应的待选物理计划进行代价估算，从待选计划中选择代价最低的物理计划作为最终的执行计划

————— SQLite主要采用RBO/CBO —————



SQLite查询重写



- 子查询扁平化和其他启发式重写
 - 子查询转连接
- 谓词逻辑重写
 - $R.x < 10 \text{ AND } R.x = S.y \rightarrow S.y < 10$
- 常量运算表达式
 - $R.x < 10 + 2 + R.y \rightarrow R.X < 12 + R.Y$
- 如果没有索引，则检查是否在这个表上对rowid的查询条件。
- 如果有索引，则统计每个表的索引信息
- 如果WHERE子句中存在OR操作符，那么要把这些OR连接的所有子句分开再进行分析



SQLite查询优化

□ 代价估计

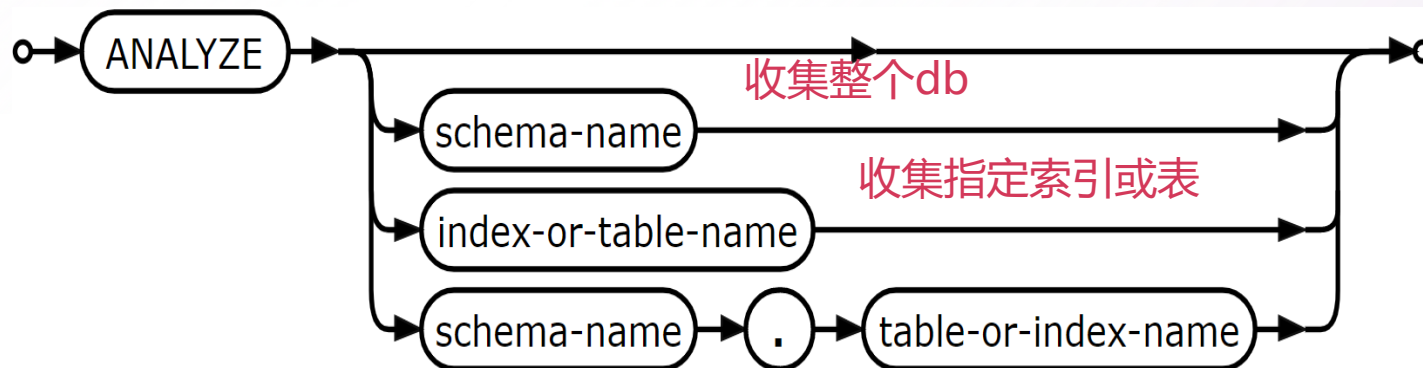
- 对查询计划中的基表、连接的结果进行基数估计
- 在基数的基础上计算执行代价

□ 总代价=IO代价+CPU代价

- 处理页面产生IO代价
- 处理元组（表达式计算）产生CPU代价

□ IO和CPU代价的衡量

- 查询所读取的记录数量
- 结果是否需要排序
- 是否需要访问索引和原表





SQLite查询优化



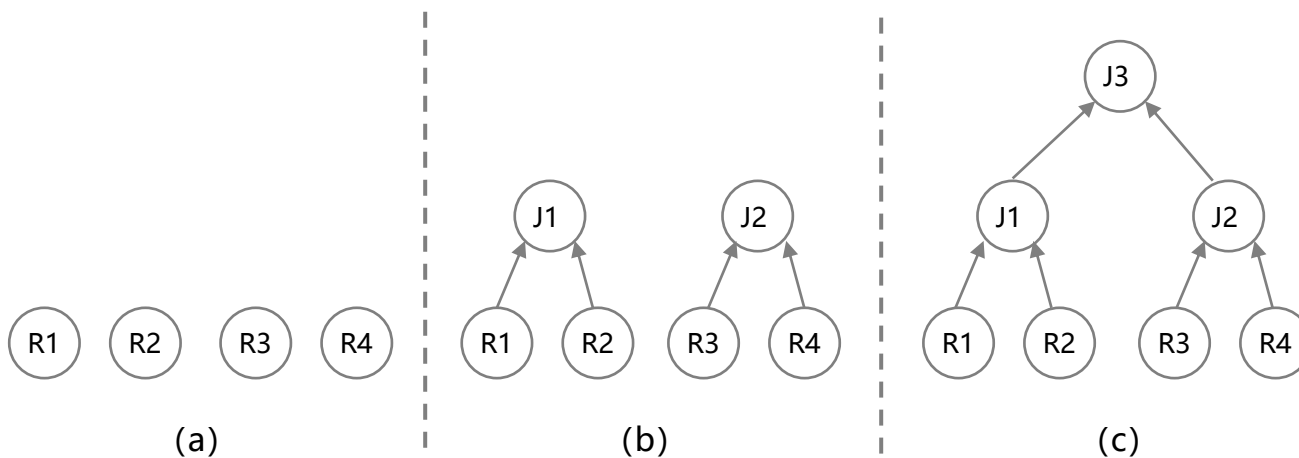
物理计划选择采用的搜索策略：

□ 自底向上模式

- (a) 确定每个表的物理扫描算子
- (b) 确定物理连接算子
- (c) 确定完整的物理执行计划

□ 不同的扫描和连接算子会生成多个候选执行计划

□ 根据各个执行计划的估算代价，选择出代价最低的执行计划





SQLite查询执行

□ 虚拟机思想

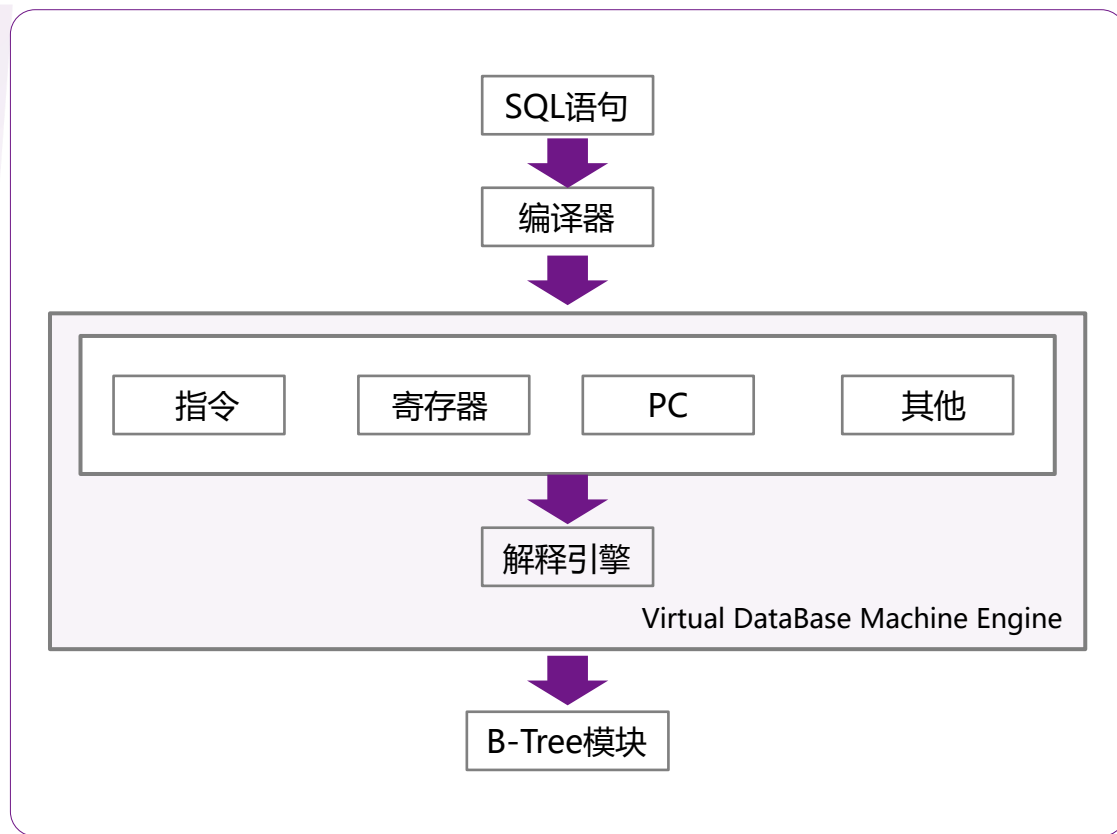
- SQL语句经过查询分析和优化，生成执行计划的字节码
- 在虚拟机中运行字节码程序，得到给定查询的执行结果

□ 字节码程序

- 由一个或多个指令组成，每个指令都有一个操作码和五个操作数

□ 字节码虚拟机 (Virtual DataBase Engine)

- 从指令编号0开始执行
- 遇到Halt指令，或者程序计数器大于最后一条指令的地址，执行停止

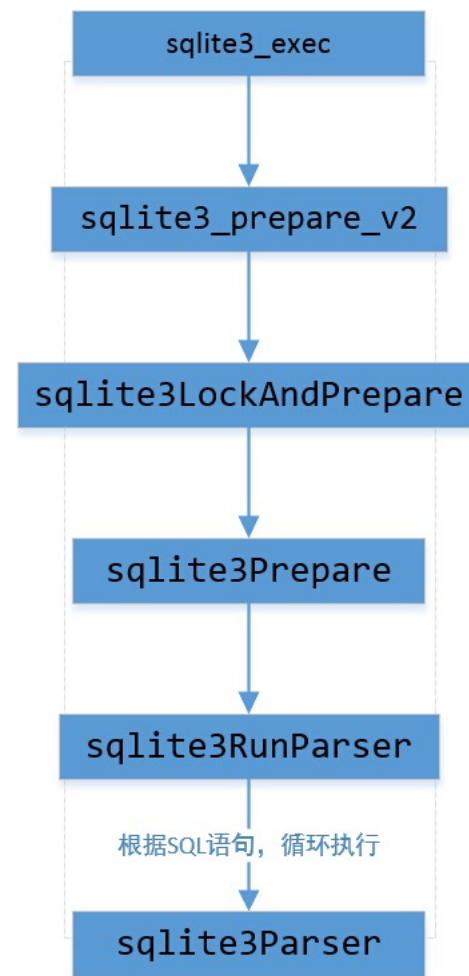




SQLite字节码虚拟机



- ❑ VDBE 虚拟数据库引擎(virtual database engine)
- ❑ 对真实计算机资源环境的一个抽象。
- ❑ 原理类似于Java中的JVM (java虚拟机) , 实现跨平台
- ❑ 流程:
 - 将SQL语句编译为虚拟机指令
 - 虚拟机执行指令 (调用B-tree相关的接口)
- ❑ SQLite会根据不同平台实现虚拟机,
- ❑ VDBE有自己完善的硬体架构, 如处理器、堆栈、寄存器等, 还具有相应的指令系统, VDBE屏蔽了与具体操作系统平台相关的信息, 使得程序只需生成在虚拟机上运行的目标代码 (字节码), 就可以在多种平台上不加修改地运行。





目录

01

SQLite嵌入式数据库

02

SQLite总体架构

03

SQLite查询处理技术

04

SQLite存储技术

05

SQLite事务管理技术

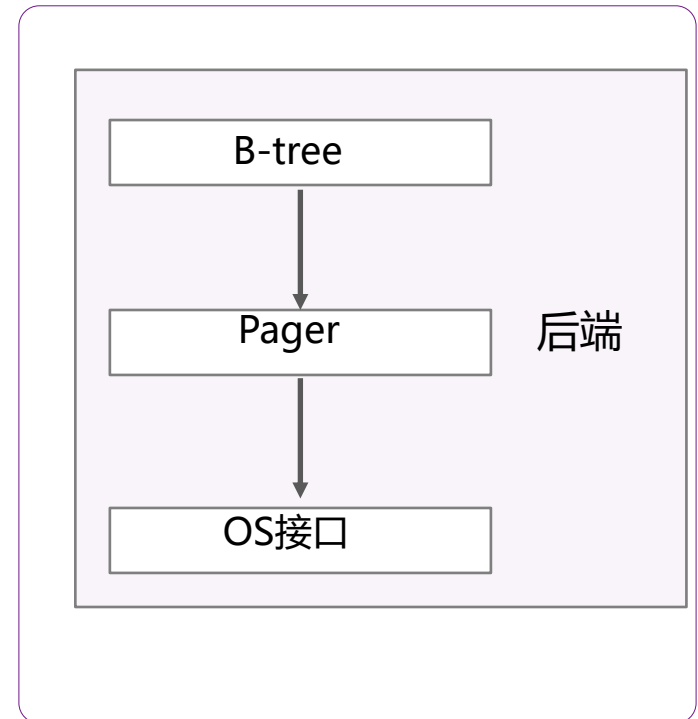


SQLite存储概览

□ 后端由三个模块组成

- B-Tree: 存储引擎的核心，将所有数据组织为树形式存储
- Pager: 以页面为单位完成各种操作
- OS Interface: 包含面向不同操作系统的底层函数

□ SQLite是典型的行存储引擎，对于内存数据库没有特别的支持。使用“:memory:”的特殊文件名可以创建一个构架完全相同、但是存放在内存中的数据库。





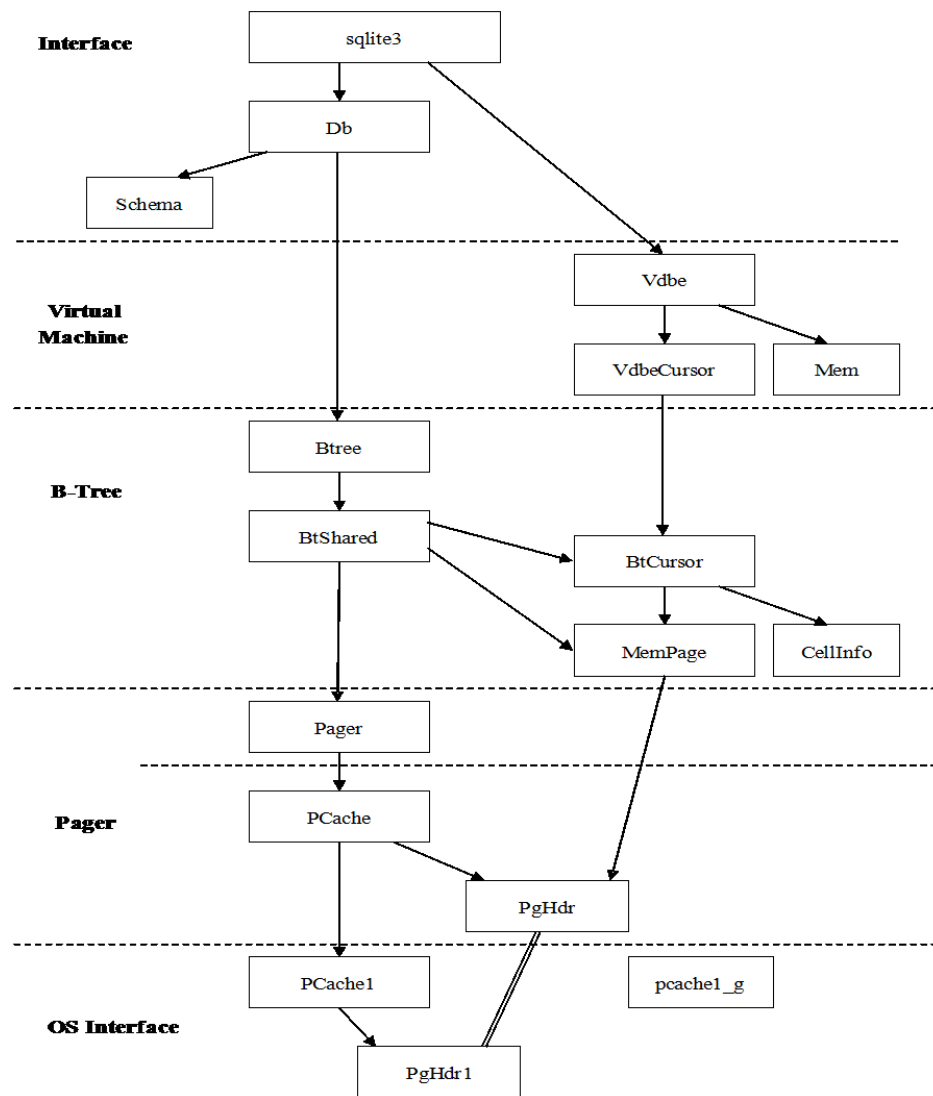
SQLite的页面结构

□ 页面的大小必须是2的幂次字节

- 更细节的说, 页面大小还应当介于512~65536之间
- SQLite诞生的时候, 默认页面大小是1024字节, 目前已经增加至4096字节

□ SQLite使用了两种B树变体: B树和B+树

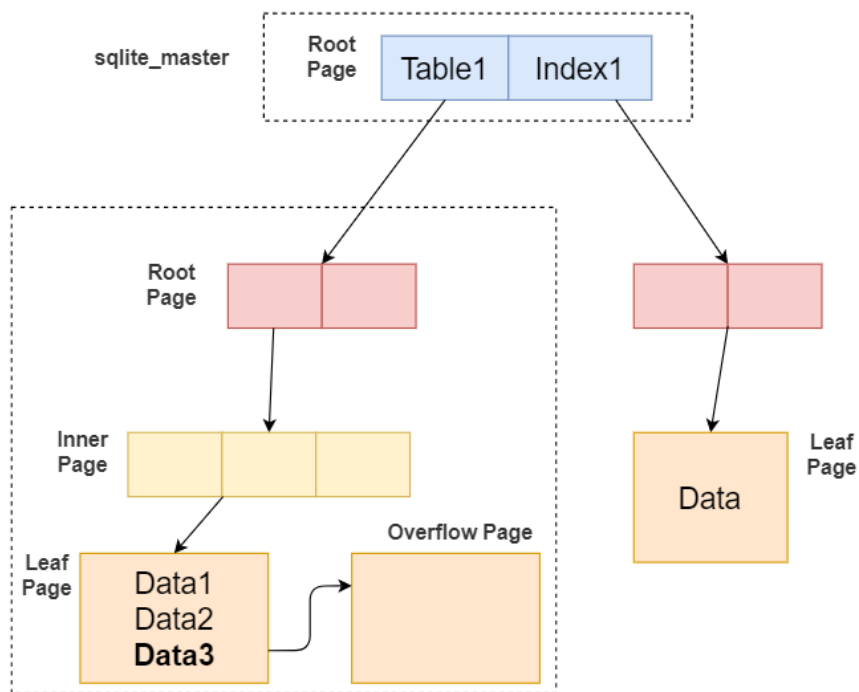
- 基于B树的索引组织表
- 如果没有主键, rowid是隐藏主键整数自增列
- 表B树: 使用整数作为键, 并将所有数据存放在叶子节点中
- 索引B树: 键是任意的, 所有信息储存在键中, 不包含额外的数据



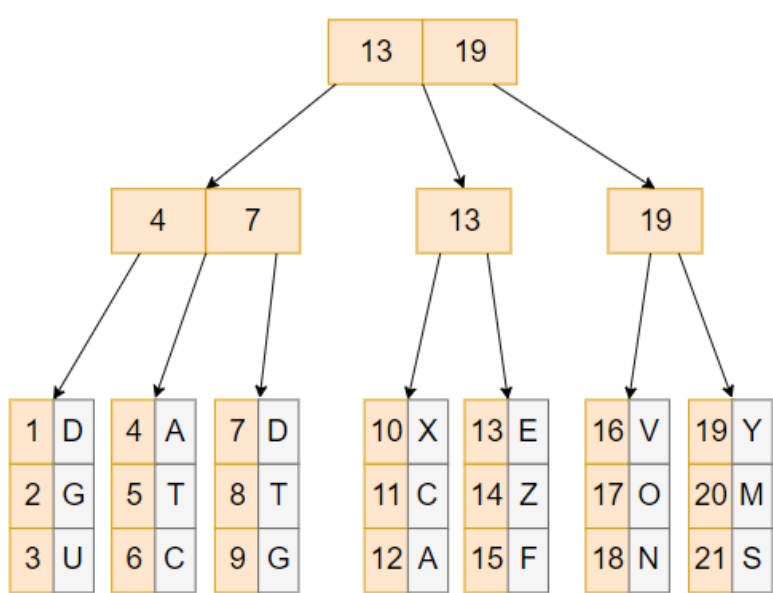


SQLite页面结构

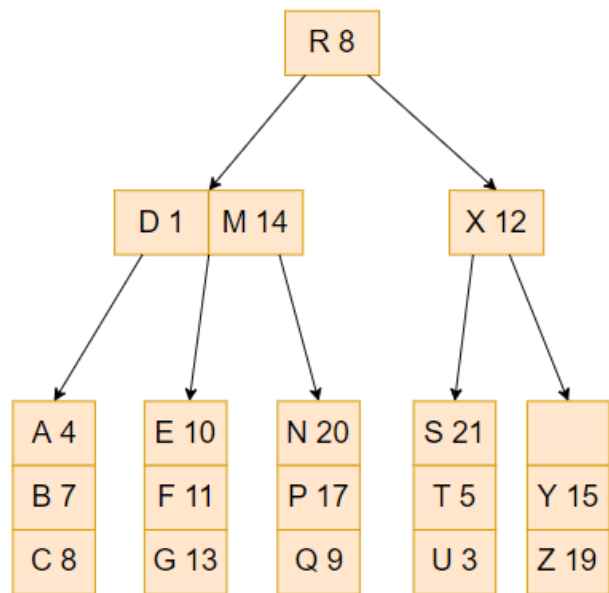
- ❑ SQLite是文件型数据库，其中数据库文件被分成大小相同的Page进行管理；
- ❑ 页面采用B树来管理存储，每个表和索引对应一棵B树；
- ❑ sqlite_master表存储Btree的元数据和根页面位置；
- ❑ 表B树中间节点不存储数据，索引B树中间节点会存储数据；



SQLite数据组织形式



表B树



索引B树



B树页面结构



□ B树的页面有两种类型

- 叶子页面：存储键和数据（对于表B树）。
- 内部页面：除了键和数据，还包含指向其子节点的指针。在这里指针仅仅是一个代表地址的整数。

□ B树使用单元格(cell)为基本单位

- Cell 指针和Cell值（类似于槽）
- Cell数组中的元素，是按照key顺序排列的
- 对于叶子页面：存储键和数据放在单元格中。
- 对于内部页面：存储键、孩子指针、和数据
 - B+树内部节点不存数据



页面类型的标志位
cell数组的大小
右子树页面的页面
编号

一个典型的B树页面结构

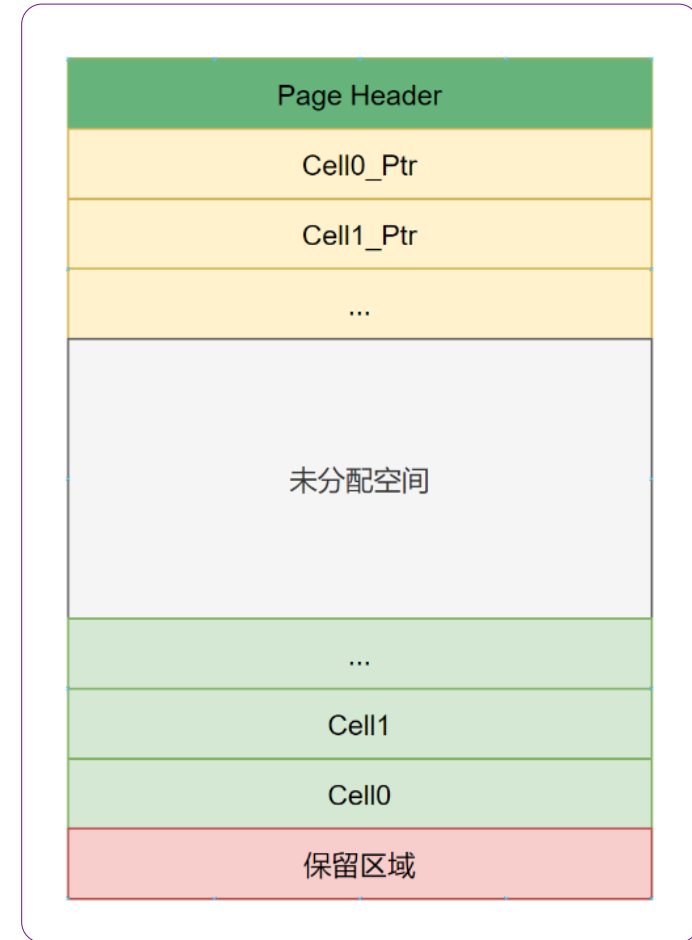


B树页面结构

□ 实际上的B树页面按照右图所示组织数据

□ 首先是8-12字节的页面头，包含以下信息：

- B树页面类型
- 空闲块的地址和碎片空间
- 页面中包含的单元格数量
- 页面大小
- 最右侧的指针（对于内部页面）
- 有孩子的12字节，没左孩子的8字节



一个典型的B树页面结构



B树页面结构

- ❑ Cell指针：紧跟的是单元格指针数组区域，每个单元格的页内偏移量会用2个字节表示。由于它们定长、连续，所以可以使用分治算法来进行查找。
- ❑ 页面底部有一个保留区域，其可能有一些复杂的用途（例如加密、校验）。也可以被设定为0字节。
- ❑ Cell数据：保留区域上方是单元格内容区，SQLite努力将单元格放置在尽可能靠近B树页面末尾的位置，以便为单元格指针数组的未来增长留出空间。

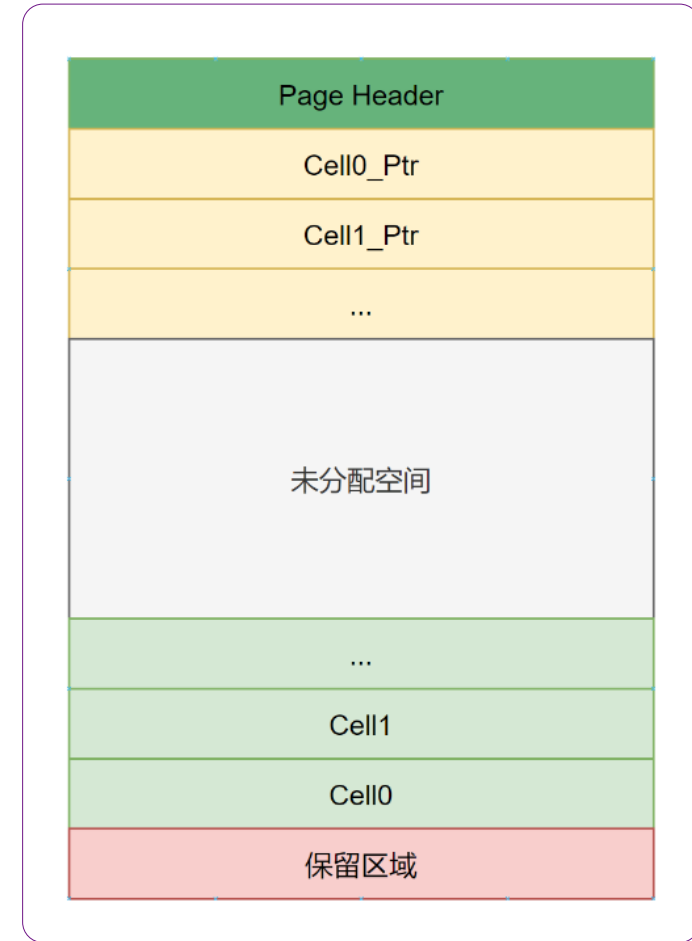


一个典型的B树页面结构



B树页面结构

- 空闲区域：最后一个单元格指针和第一个单元格数据的开始之间是未分配区域。
- 页面中所有连续的大于4字节的空闲空间组织成一个链：
 - 每个空闲块的前4个字节会标识空闲块的大小和下一个空闲块的偏移量。
 - 小于4字节的孤立空闲空间被称为碎片，一般不能超过60字节。



一个典型的B树页面结构



B树页面组织

□ 在SQLite中，数据库中的每一张表都使用B树表示

- 对于一张表，会使用SQLite分配的64位带符号整数rowid作为键的表B树表示
- 对于一个索引，会使用索引列和对应表行的rowid作为键的索引B树表示

□ 还有一种WITHOUT ROWID表，在这种表中使用用户声明的主键而非自动分配的rowid作为行的标识

- 在SQLite内部会使用索引B树而非表B树来表示
- 一般来说其性能会弱于常规表，但在一些具有较长字符串主键的表中可能运行较快



元数据页面组织

□ 每个SQLite数据库都包含一张sqlite_schema系统表，其包含数据库中全部的：

- 表
- 索引
- 视图
- 触发器

□ 系统表中保存上述这些模式对应的B树的根节点来提供对它们的访问路径。而sqlite_schema表是数据库第一张表，紧跟在100字节的数据库头后方。

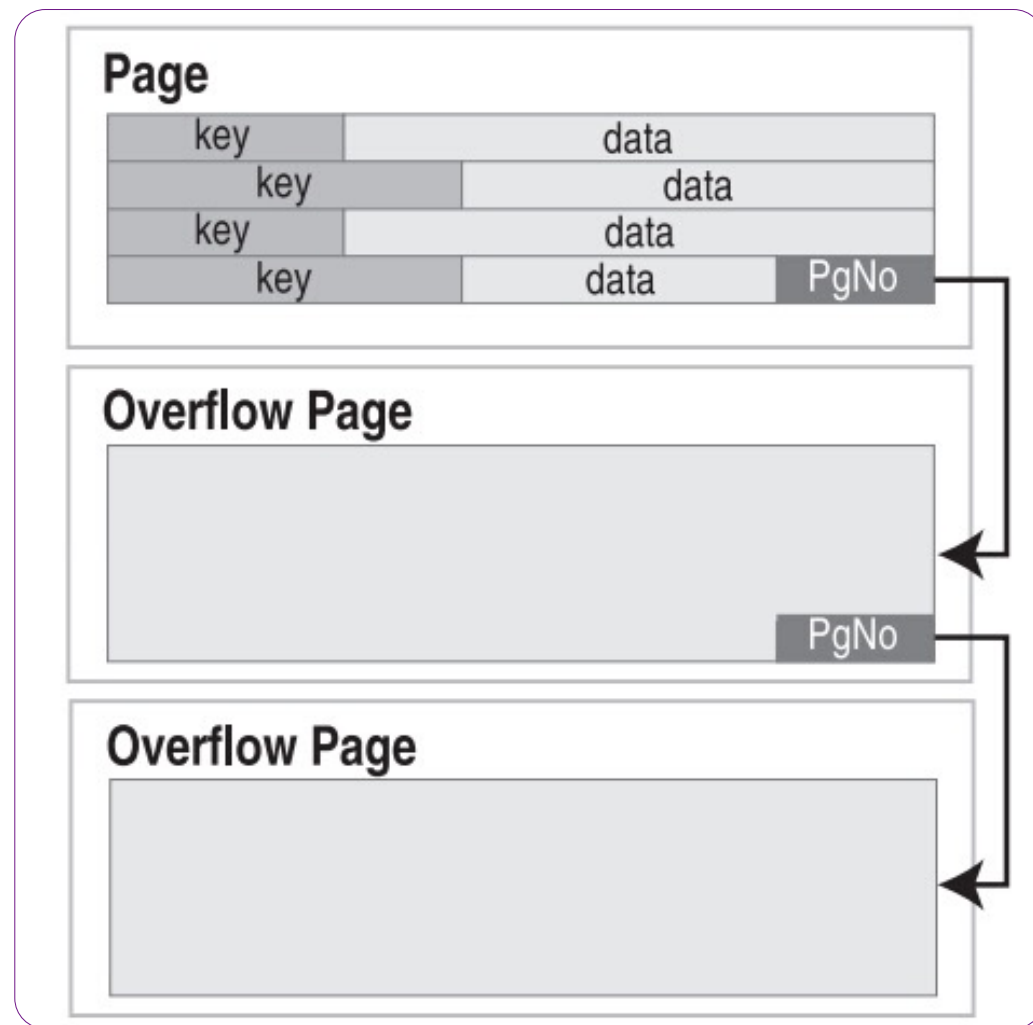


页面溢出

□ SQLite中的页面类型包括：

- 叶子页面(leaf): 存储数据
- 内部页面(internal): 包含查询时的导航信息
- 溢出页面(overflow)
- 空闲页面(free)
 - trunk pages (主页面)
 - leaf pages(叶子页面)

□ Freelist (空闲页面链表)





目录

01

SQLite嵌入式数据库

02

SQLite总体架构

03

SQLite查询处理技术

04

SQLite存储技术

05

SQLite事务管理技术



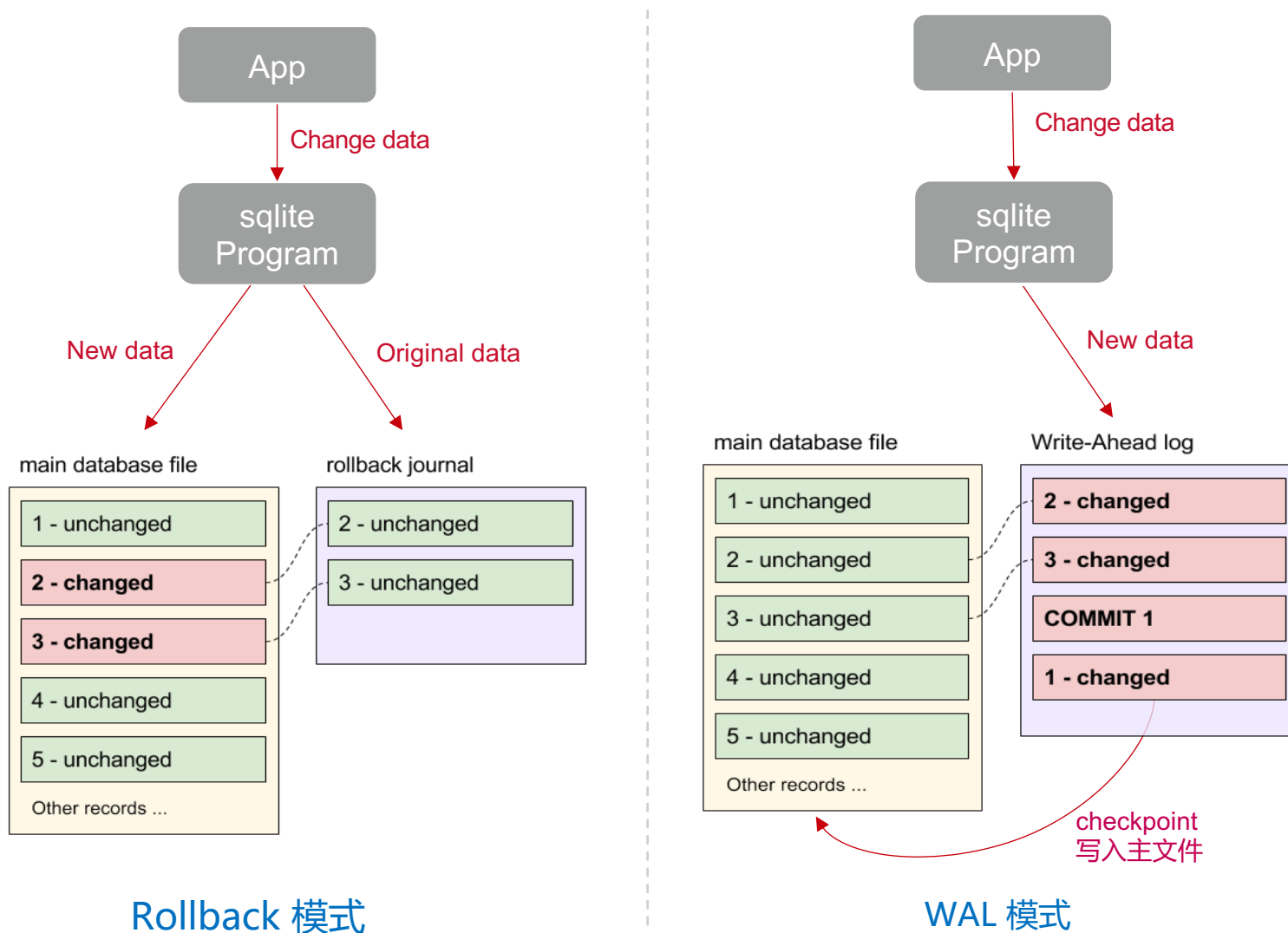
SQLite事务管理技术

- SQLite使用和数据库文件在同一目录的临时日志文件实现事务的原子性、隔离、并发控制和故障恢复等，以保证事务的ACID原则，其有两种主要的实现模式：
 - 支持完整ACID事务，采用文件锁(库级)模型，隔离级别做到串行化；
 - **回滚模式 (Rollback mode)**
 - 将要更改的部分数据库文件复制到回滚日志文件，确保将回滚日志文件刷入磁盘，然后再直接对原数据库文件进行更改
 - **预写日志模式 (Write-Ahead Log mode, WAL mode)**
 - 与回滚模式相反，原始内容将保留在原数据库文件中，更改将附加到单独的WAL文件中

————— 回滚模式为默认方式 —————



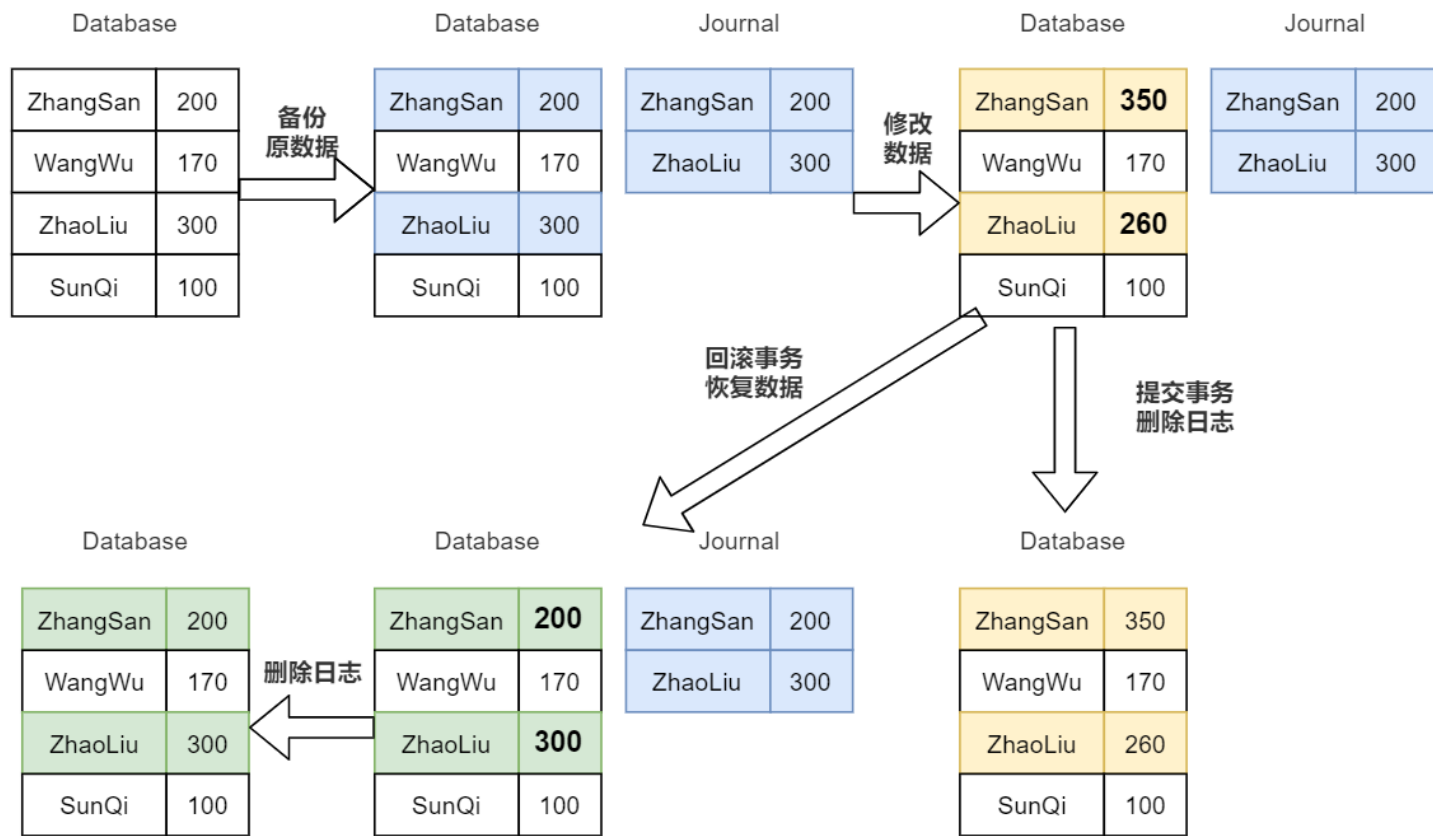
SQLite事务管理技术





回滚模式 (Rollback)

- ❑ 1. 先将原始页面备份到journal文件中，然后修改数据库页面；
- ❑ 2. Commit时将journal文件删除；
- ❑ 3. Rollback时将journal文件恢复到数据库文件中；





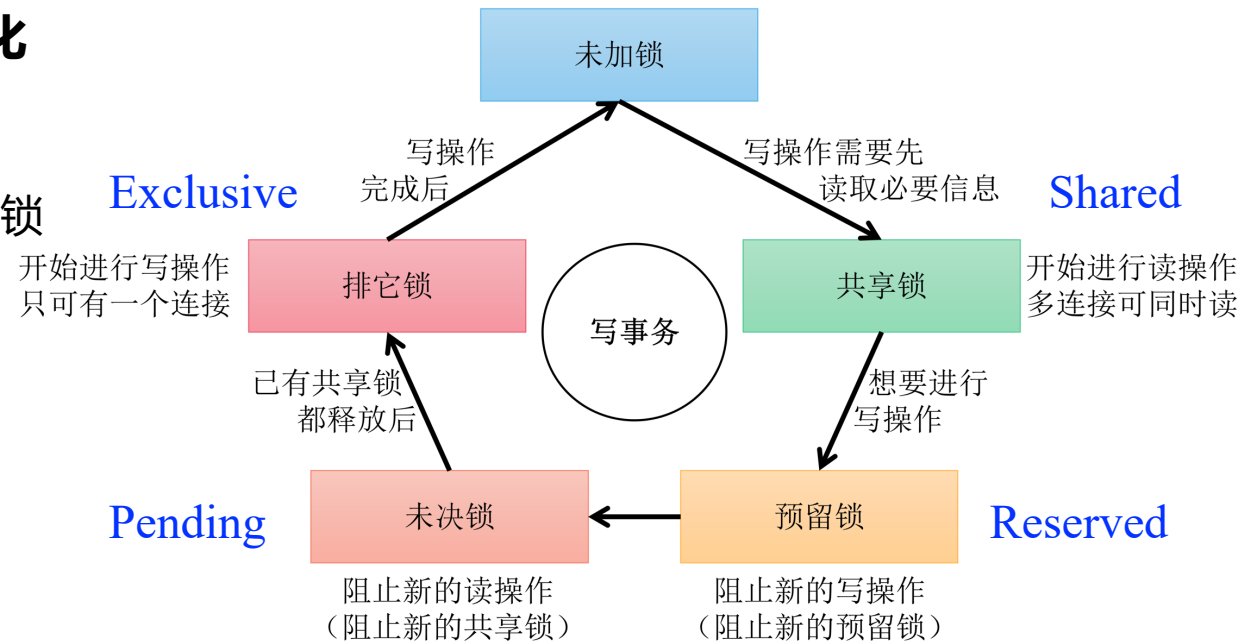
回滚模式

□ 锁

- 读数据时，尝试获取Shared锁，获取读锁之后读取数据；
- 写数据时，锁状态：无锁->共享锁->保留锁->未决锁->独占锁
- SQLite中有五种锁状态：未加锁、共享锁、预留锁、未决锁和排它锁

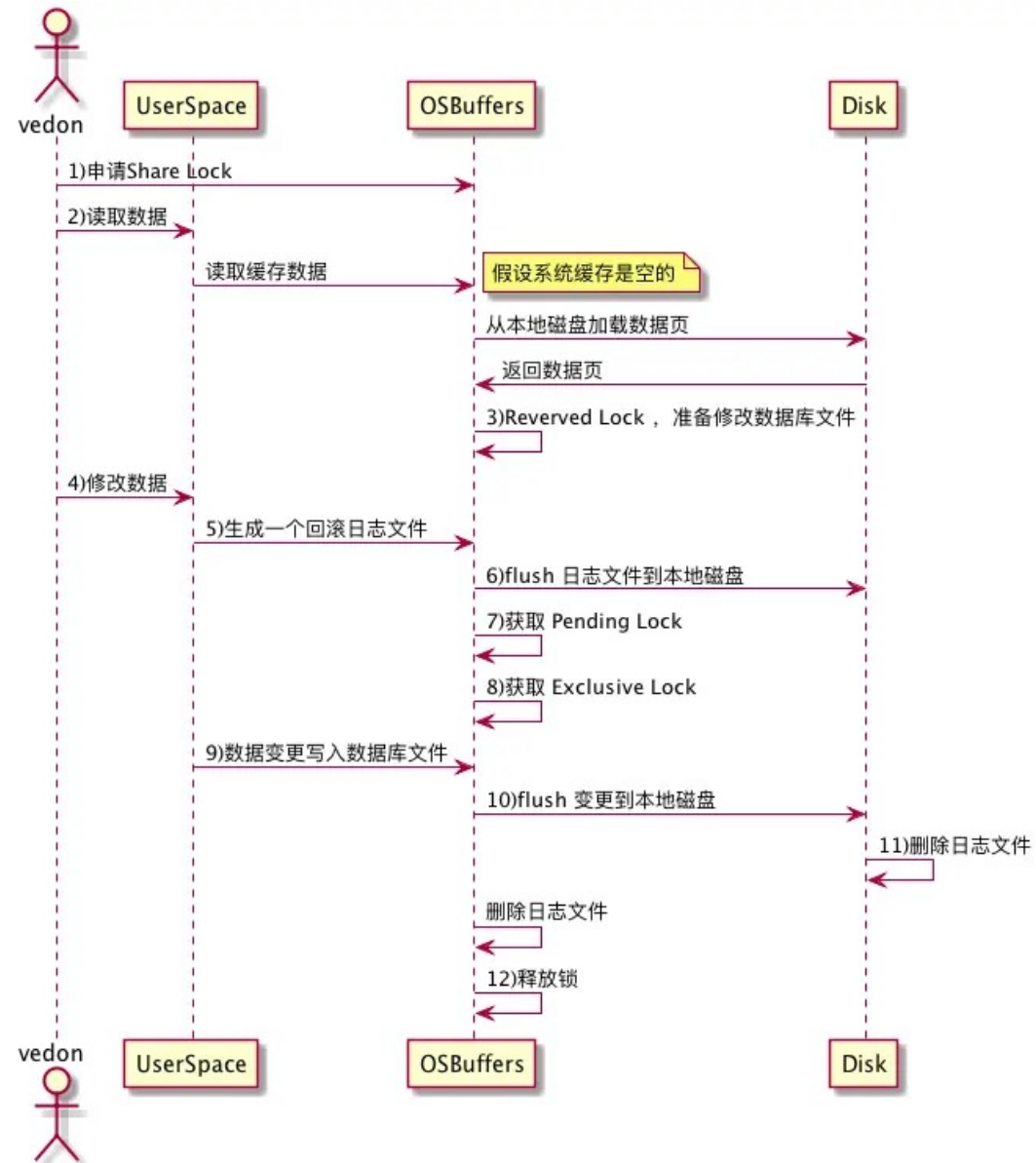
– 回滚模式下，如果要进行写事务，锁会经历五次变化

- 首先假设当前数据库处于未加锁状态
- 写操作也需要读取一些必要的信息，即需要获取共享锁
- 然后升级为预留锁，阻止新的写操作
- 再升级为未决锁，阻止新的读操作
- 真正进行写操作，需升级为排它锁
- 修改完成后释放锁





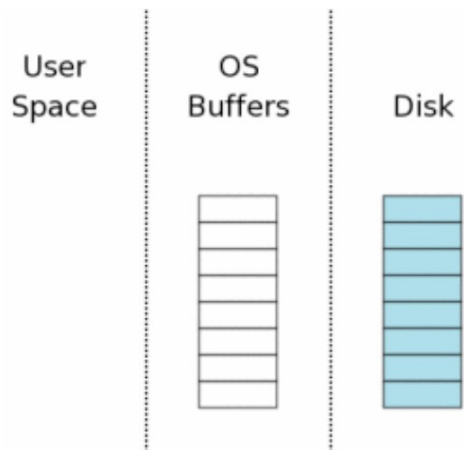
回滚模式流程



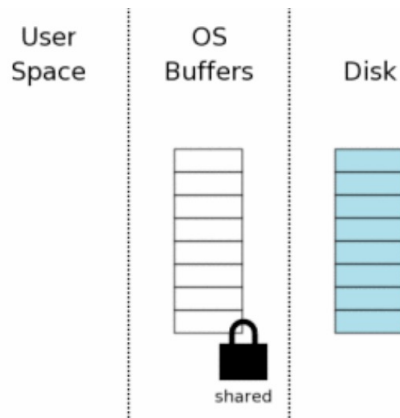


回滚模式 - 案例

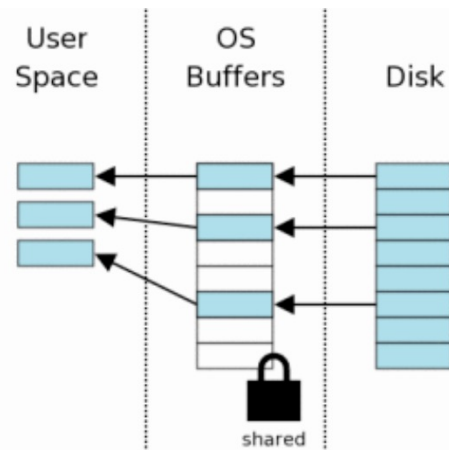
① 初始状态



② 请求读数据：申请共享锁

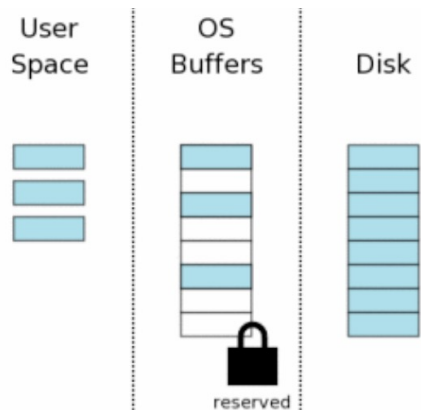


③ 读数据

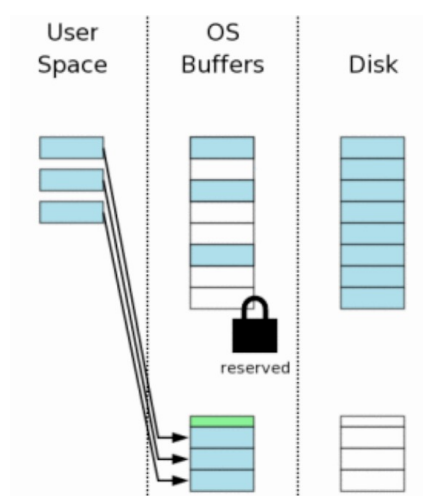


④ 请求写数据：申请Reserved锁

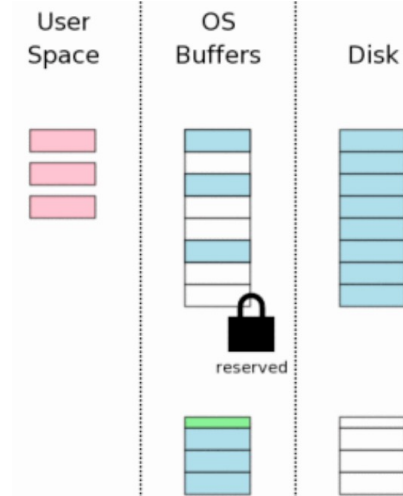
与其他共享锁相容。但同时只能存在一个Reserved



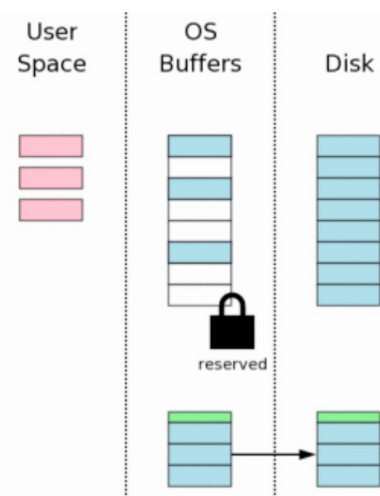
⑤ 生成回滚日志



⑥ 修改用户本地数据



⑦ 回滚日志写盘

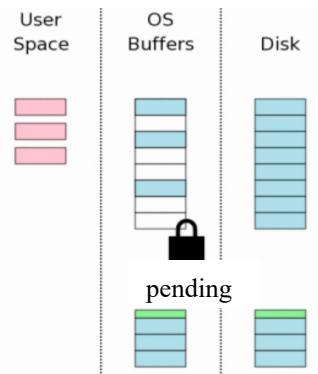




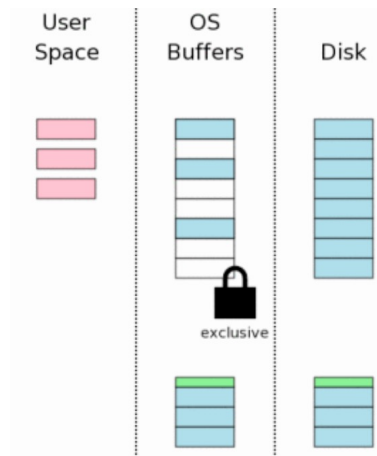
回滚模式 – 案例

⑧ 请求写数据：申请Pending 锁

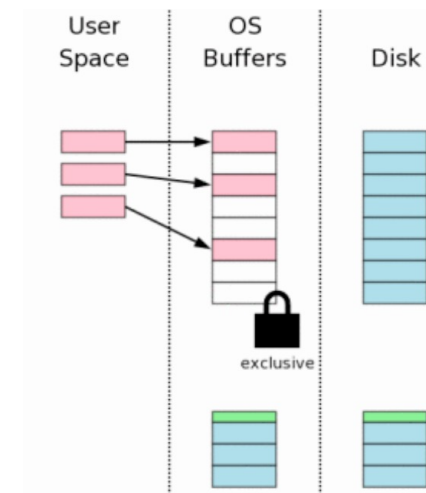
允许已经获得一个共享锁的进程继续读取数据，但阻止新的共享锁的授予。



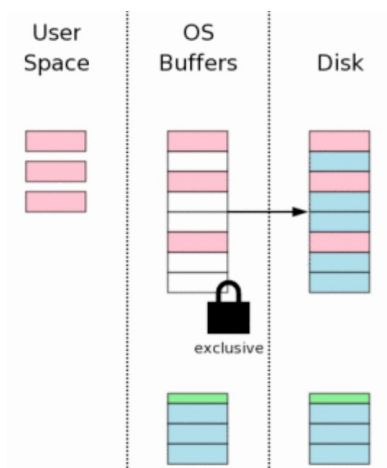
⑨ 请求写数据：申请Exclusive锁



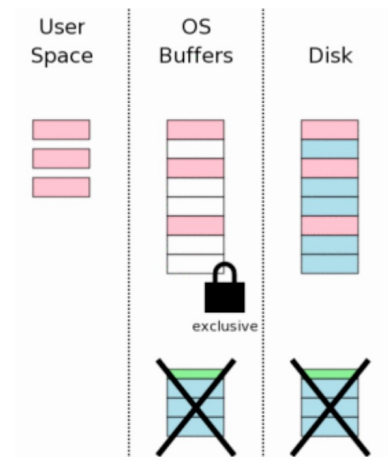
⑩ 写数据到缓存



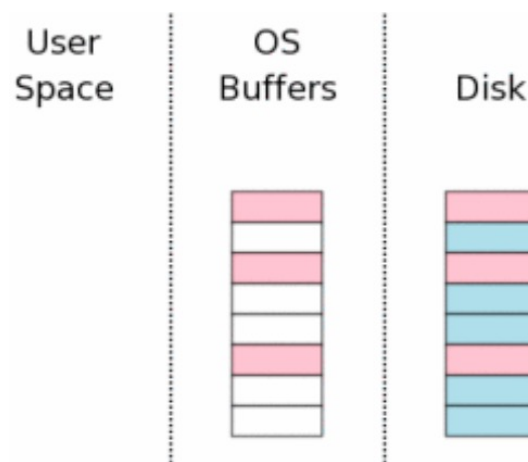
⑪ 刷新到磁盘



⑫ 删除回滚日志



⑬ 释放锁

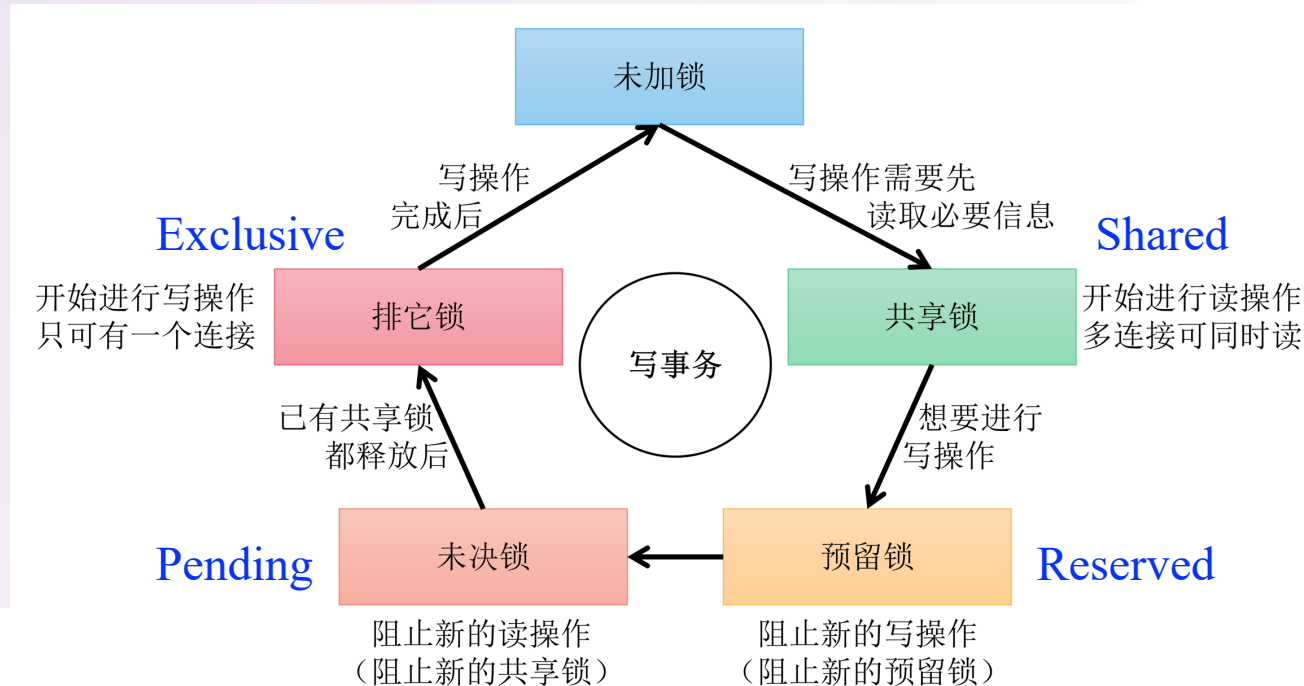




回滚模式 – 恢复

□ 回滚日志文件

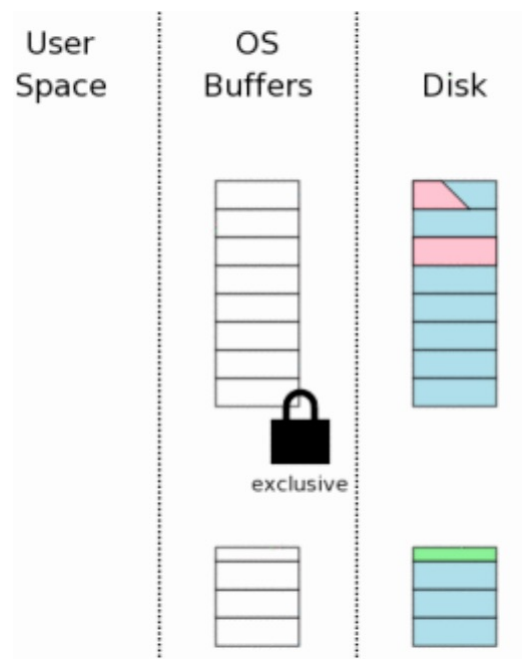
- 预留锁时期，创建单独的回滚日志文件，将原始数据写入回滚日志文件，然后对用户进程空间的页面进行修改，并确保将日志刷入磁盘
- 排它锁时期，真正将修改写入磁盘，即把用户进程空间的已修改的页面刷入磁盘。然后就可以将磁盘中的回滚日志文件删除或将回滚日志头文件清零
- SQLite会根据回滚日志文件是否存在判断是否进行恢复操作



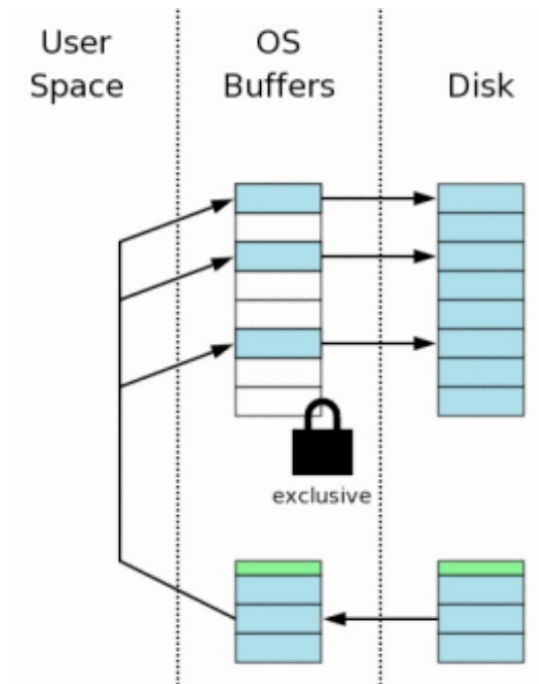


回滚模式 – 恢复

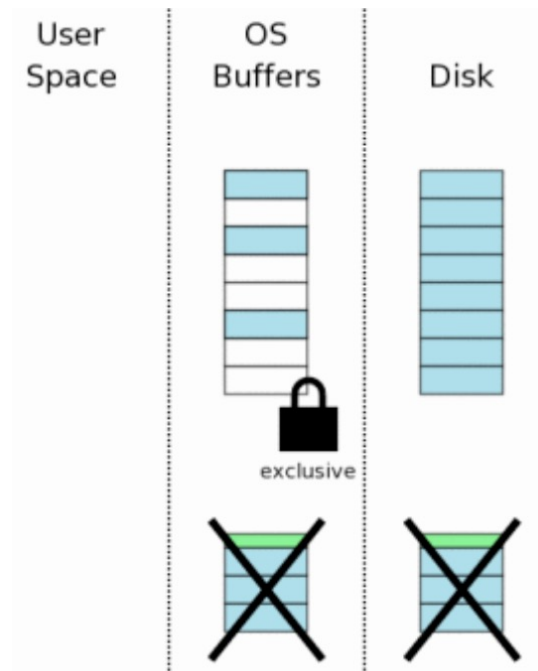
① 获取exclusive锁



② 通过回滚日志恢复



③ 删除回滚日志



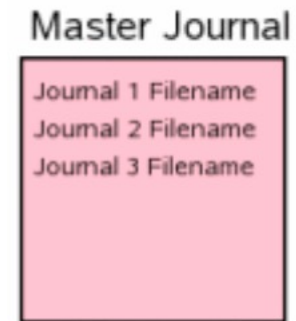
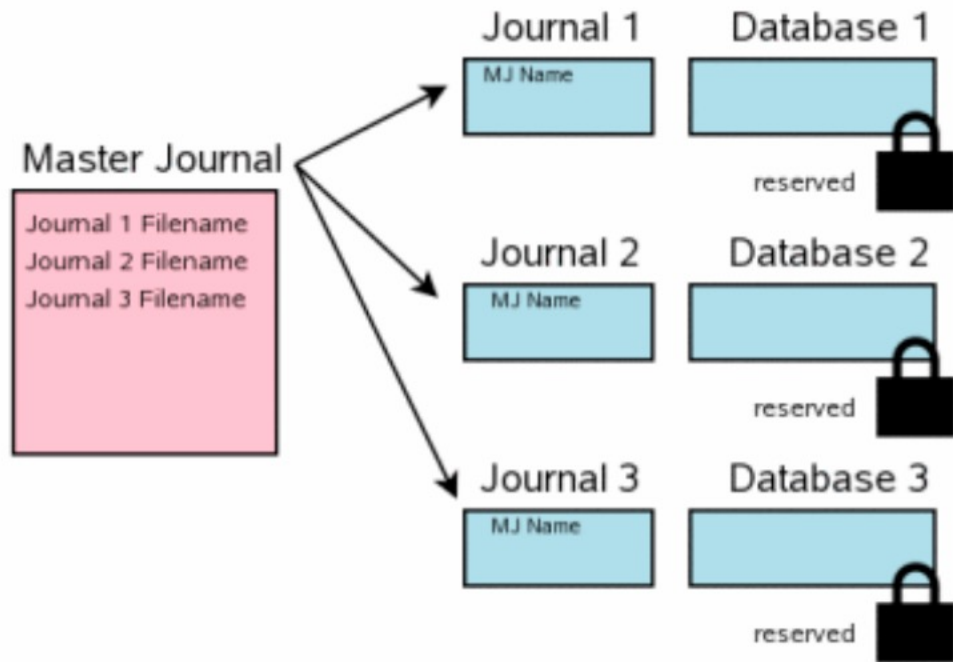


回滚模式 – 多数据库

- SQLite还允许单个数据库连接通过使用attach database命令与多个数据库进行通信
- SQLite在多个数据库之间执行事务和在单个数据库中实现思路基本一致，但更为复杂：
 - 除了回滚日志文件，还会创建超级日志
 - 超级日志仅包含每个回滚日志的完整路径名
 - 回滚日志的开头会记录超级日志的完整路径名
 - 修改写入数据库后，在删除回滚日志前需要删除超级日志
 - 多数据库事务中，在删除超级日志后即使回滚日志还没有删除就发生故障，SQLite也不会因为回滚日志存在就进行恢复操作



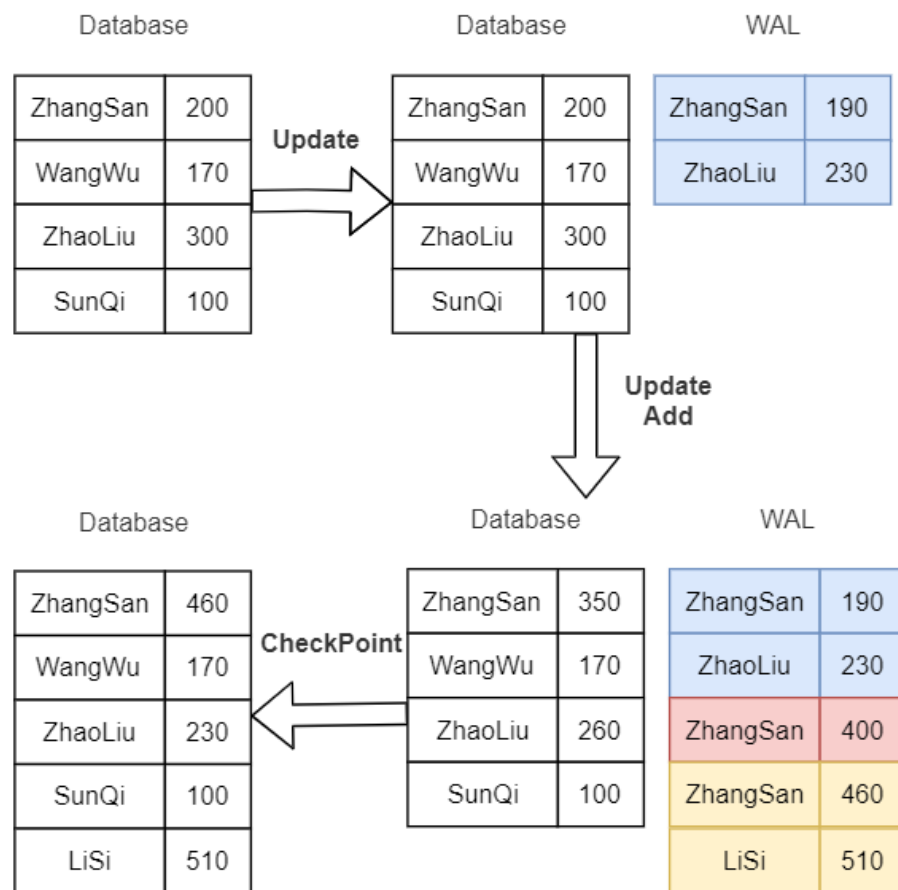
回滚模式 – 多数据库





WAL模式

- ❑ WAL模式下数据库包含：**db**、**db-wal**(日志)、**db-shm**(WAL索引)
- ❑ 写入数据时先将修改的数据以**追加**方式写入到WAL；
- ❑ WAL模式下每修改一条就会追加相关的页面到日志中，WAL会变越来越大，需要通过**Checkpoint**来同步数据到主DB中；
- ❑ WAL可以通过WAL-index加速查找





WAL模式

□ 读操作

- 连接首先会记住WAL中最后一个有效提交记录的位置，这一点叫做终点
- 对于单个读操作，读的过程中终点保持不变
- 读某一个页面时，先检查WAL中是否有该页，有则读取WAL中最新的页，没有则读取原数据库文件

□ 写操作

- 只需要在WAL文件末尾附加新的内容
- 当有一个指示提交的特殊记录附加到WAL文件时，就会进行commit
- WAL模式下写操作不会干扰到读操作，故读写可以同时进行

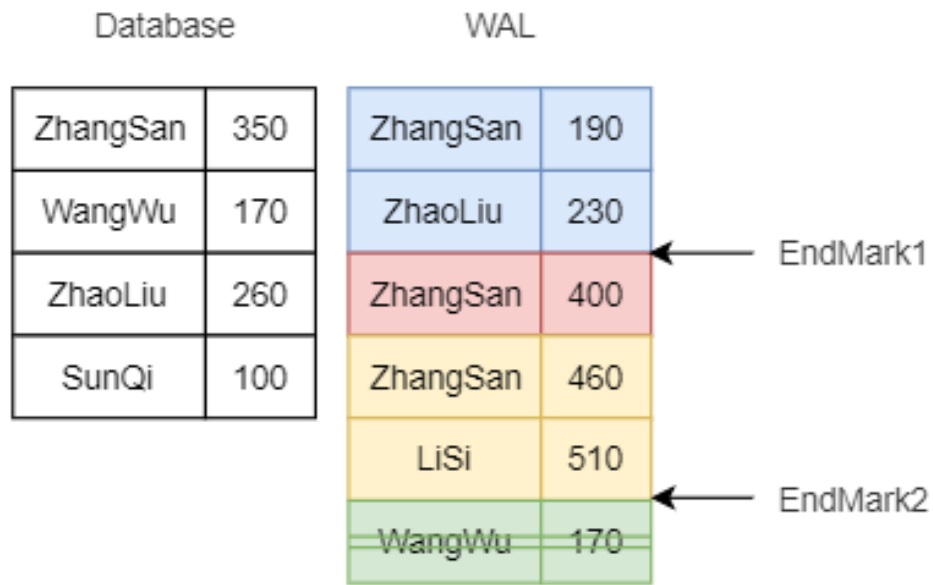
□ checkpoint操作

- 该操作会将WAL文件中的页面移到原始数据库中

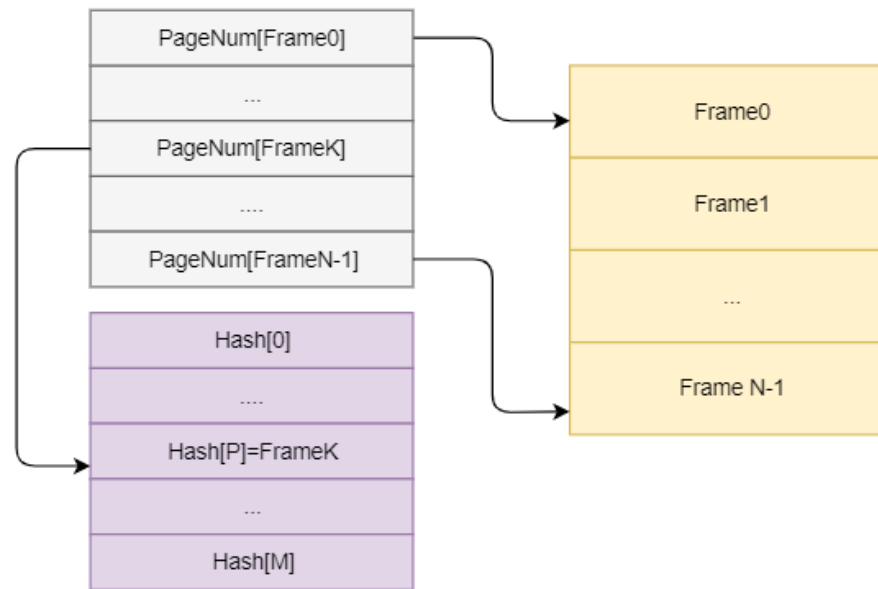


WAL模式

- ❑ 读取数据时，优先查找WAL文件中的数据(WAL文件内容比DB文件的数据更新)，如果在WAL中查到，则不再读取主DB文件；
- ❑ WAL文件变大时，如果遍历读取数据时就会出现效率低；SQLite使用WAL-Index文件来辅助获取到待读数据的最新页面；



WAL模式读事务机制



SQLite WAL-Index使用



两种模式对比

□ WAL模式相比于回滚模式主要优点

- WAL模式写操作非常快，因为只需要写一次内容
- WAL模式写操作是连续的
- WAL提供了更好的并发性，读和写可以同时进行

□ WAL模式相比于回滚模式主要缺点

- 随着WAL文件大小的增加，读性能会下降，读多写少时WAL模式可能比回滚模式慢一些
- WAL模式不能在网络文件系统上工作



SQLite小结

01

SQLite嵌入式数据库

02

SQLite总体架构

03

SQLite查询处理技术

04

SQLite存储技术

05

SQLite事务管理技术